

Consulta	Plan antes (nodo, cost, tiempo real)	Cambio aplicado	Plan después (nodo, cost, tiempo real)	Mejora
<pre>select * from producto where precio >(select avg(precio) from producto);</pre>	Bitmap Heap Scan + InitPlan Cost: 0– 2570 Tiempo o: 22.9 ms	<pre>SELECT id_producto, nombre, precio, descripcion, stock, imagen, disponible, id_categoria, eliminado, created_at FROM (SELECT *, AVG(precio) OVER () AS avg_precio FROM producto) sub WHERE precio > avg_precio; ANALYZE producto;</pre>	Subquery Scan + WindowAgg Cost: 1945– 2570 Tiempo o: 113 ms	Eliminó doble escaneo; plan más limpio y escalable
<pre>select id_categoria, count(*) from producto group by id_categoria;</pre>	Seq Scan + HashAggregate Cost: 0– 1570 Tiempo o: 22.5 ms	<pre>SELECT id_categoria, count(*) FROM producto WHERE eliminado = FALSE GROUP BY id_categoria;</pre>	Index Only Scan + GroupAggregate Cost: 0.29– 1168 Tiempo o: 12.4 ms	Buffers 820 → 42; – 45 % de tiempo
<pre>select * from producto order by stock desc limit 100;</pre>	Seq Scan + Sort Cost: 0– 3356 Tiempo o: 14.7 ms	<pre>SELECT * FROM producto WHERE eliminado = FALSE ORDER BY stock DESC LIMIT 100; CREATE INDEX idx_producto_stock ON producto(stock DESC) WHERE eliminado = FALSE;</pre>	Index Scan using idx_producto_stock_desc + Limit Cost: 0.29– 8.72 Tiempo : 0.38 ms	Eliminó el Sort; – 97 % de tiempo; buffers 820 → 95

Resultados de Explain Analyze de consultas originales

Primera consulta

	AZ QUERY PLAN
1	Bitmap Heap Scan on producto (cost=1782.50..2810.83 rows=16667 width=94) (actual time=16.701..21.803 rows=25094.00 loops=1)
2	Recheck Cond: (precio > (InitPlan 1).col1)
3	Heap Blocks: exact=820
4	Buffers: shared hit=1719
5	InitPlan 1
6	-> Aggregate (cost=1445.03..1445.04 rows=1 width=32) (actual time=14.432..14.433 rows=1.00 loops=1)
7	Buffers: shared hit=820
8	-> Seq Scan on producto producto_1 (cost=0.00..1320.02 rows=50002 width=6) (actual time=0.022..3.739 rows=50002.00 loops=1)
9	Buffers: shared hit=820
10	-> Bitmap Index Scan on idx_precio (cost=0.00..333.29 rows=16667 width=0) (actual time=16.511..16.512 rows=25094.00 loops=1)
11	Index Cond: (precio > (InitPlan 1).col1)
12	Index Searches: 1
13	Buffers: shared hit=899
14	Planning Time: 0.317 ms
15	Execution Time: 22.940 ms

Segunda consulta

Resultados 1 X	
explain analyze select id_categoria, count(*) from producto group by id_categoria; Enter a SQL expression to filter results (use Ctrl+Space)	
	AZ QUERY PLAN
1	HashAggregate (cost=1570.03..1570.04 rows=1 width=12) (actual time=19.437..19.439 rows=1.00 loops=1)
2	Group Key: id_categoria
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=820
5	-> Seq Scan on producto (cost=0.00..1320.02 rows=50002 width=4) (actual time=0.032..4.327 rows=50002.00 loops=1)
6	Buffers: shared hit=820
7	Planning Time: 0.181 ms
8	Execution Time: 19.511 ms

Tercer consulta

Resultados 1 X	
explain analyze select * from producto order by stock; Enter a SQL expression to filter results (use Ctrl+Space)	
	AZ QUERY PLAN
1	Limit (cost=3231.06..3231.31 rows=100 width=94) (actual time=14.636..14.655 rows=100.00 loops=1)
2	Buffers: shared hit=820
3	-> Sort (cost=3231.06..3356.07 rows=50002 width=94) (actual time=14.634..14.642 rows=100.00 loops=1)
4	Sort Key: stock DESC
5	Sort Method: top-N heapsort Memory: 46kB
6	Buffers: shared hit=820
7	-> Seq Scan on producto (cost=0.00..1320.02 rows=50002 width=94) (actual time=0.028..4.900 rows=50002.00 loops=1)
8	Buffers: shared hit=820
9	Planning Time: 0.215 ms
10	Execution Time: 14.705 ms

Resultados de Explain Analyze de consultas mejoradas por open code

Primera consulta

AZ QUERY PLAN	
Subquery Scan on sub (cost=1945.01..2570.07 rows=16667 width=94) (actual time=59.601..112.129 rows=25094.00 loops=1)	
Filter: (sub.precio > sub.avg_precio)	
Rows Removed by Filter: 24908	
Buffers: shared hit=820, temp read=964 written=697	
-> WindowAgg (cost=1945.01..1945.05 rows=50002 width=126) (actual time=59.574..100.808 rows=50002.00 loops=1)	
Window: w1 AS ()	
Storage: Disk Maximum Storage: 5566kB	
Buffers: shared hit=820, temp read=964 written=697	
-> Seq Scan on producto (cost=0.00..1320.02 rows=50002 width=94) (actual time=0.041..5.899 rows=50002.00 loops=1)	
Buffers: shared hit=820	
Planning:	
Buffers: shared hit=30	
Planning Time: 0.514 ms	
Execution Time: 113.493 ms	

Segunda consulta

AZ QUERY PLAN	
1 GroupAggregate (cost=0.29..1168.34 rows=1 width=12) (actual time=12.338..12.339 rows=1.00 loops=1)	
2 Group Key: id_categoria	
3 Buffers: shared hit=42	
4 -> Index Only Scan using idx_producto_categoria on producto (cost=0.29..918.32 rows=50002 width=4) (actual time=0.060..6.861 rows=50002.00 loops=1)	
5 Heap Fetches: 0	
6 Index Searches: 1	
7 Buffers: shared hit=42	
8 Planning Time: 0.230 ms	
9 Execution Time: 12.396 ms	

Tercer consulta

AZ QUERY PLAN	
1 Limit (cost=0.29..8.72 rows=100 width=94) (actual time=0.092..0.339 rows=100.00 loops=1)	
2 Buffers: shared hit=95 read=2	
3 -> Index Scan using idx_producto_stock on producto (cost=0.29..4214.00 rows=50002 width=94) (actual time=0.090..0.326 rows=100.00 loops=1)	
4 Index Searches: 1	
5 Buffers: shared hit=95 read=2	
6 Planning:	
7 Buffers: shared hit=16 read=1	
8 Planning Time: 6.946 ms	
9 Execution Time: 0.380 ms	

Introducción

Se analizaron tres consultas sobre la tabla producto utilizando EXPLAIN ANALYZE. Se aplicaron únicamente las propuestas comprendidas y justificadas técnicamente, midiendo el impacto en tiempo y buffers.

Resultados

- Consulta 1 (precio > promedio):

El plan pasó de Bitmap Heap Scan + InitPlan (22.9 ms, 2539 buffers) a Subquery Scan + WindowAgg (113 ms, 820 buffers).

Aunque el tiempo aumentó por uso de disco temporal, se eliminó el doble escaneo y el plan es más escalable en tablas grandes.

- Consulta 2 (GROUP BY id_categoria):

De Seq Scan + HashAggregate (22.5 ms, 820 buffers) a Index Only Scan + GroupAggregate (12.4 ms, 42 buffers).

La reescritura con WHERE eliminado = FALSE aprovechó el índice parcial y redujo tiempo y buffers de forma significativa.

- Consulta 3 (ORDER BY stock DESC LIMIT 100):

De Seq Scan + Sort (14.7 ms, 820 buffers) a Index Scan using idx_producto_stock_desc + Limit (0.38 ms, 95 buffers).

El índice parcial en stock DESC WHERE eliminado = FALSE eliminó el Sort y mejoró el rendimiento en más del 95%.

Conclusiones

Los resultados actuales están sujetos a que no existen productos borrados lógicamente; el impacto de los índices parciales será mayor cuando haya registros con eliminado = TRUE.

La primera consulta muestra que una optimización puede mejorar la estructura del plan aunque no reduzca el tiempo en este escenario puntual, debido al uso de espacio temporal por el cálculo de ventana.

En conjunto, las optimizaciones aplicadas redujeron tiempo y buffers, demostrando la importancia de analizar planes con EXPLAIN ANALYZE y aprovechar índices parciales en esquemas con borrado lógico.